

Acne Recognition Project Report:

Introduction:

Many individuals, particularly teenagers and young adults ages 12-24, suffer from acne. Many of these individuals do not understand what type of acne they have, nor do they know how to treat it. Different types of acne respond best to different treatments. We are concerned that people are wasting their money and are worsening their condition by using the wrong products, which are not designed for their type of acne. Our team decided that the best way to help these people is to create a system that automatically and accurately classifies acne in images, thereby identifying the type of acne the user has when they submit a picture of their acne. A prevalent model is ResNet, which can learn complex visual features from images. For our study, we are developing an image classification system to identify six types of acne: nodules, blackheads, cysts, papules, pustules, and whiteheads. We used a pre-trained model, ResNet-50, included with PyTorch for our product.

We have noticed many companies that work with this type of tool, which we are trying to develop. One of which is SPOTSCAN+ (*Spotscan+, Acne-Prone Skin Analysis Powered by A.I.*, 2018), an app that uses a model very similar to ours; however, our model, as of right now, is entirely backend code, whereas SPOTSCAN+ has a frontend that actually scans the user's face instead of uploading a picture of their acne. In the medical field, CNNs are commonly used to identify anomalies like tumors, cardiovascular issues, Alzheimer's, and more (MicroAI, 2021). This shows how effective deep learning models are for image classification.

Data:

The dataset used for classification is a public dataset from Roboflow containing 1,757 images across six classes: blackheads, cysts, nodules, papules, pustules, and whiteheads. The dataset was pre-split into training, validation, and test sets consisting of 1225 (69.8%), 260 (14.8%), and 270 (15.4%) images.

The class distribution of the dataset is well-balanced across all splits. In the training set, each class contained around 198–209 samples, while the validation and test sets each contained roughly 42–46 samples per class. This balanced distribution helps reduce bias toward any single class and supports more reliable evaluation.

One of the limitations is that each image is a close-up of acne, focusing on localized skin regions rather than the entire face. This simplifies the classification task but may limit the model's ability to generalize to real-world scenarios where acne appears within more complex visual contexts.

Before training, several preprocessing steps were applied. Training images were augmented using random horizontal flips, vertical flips, and rotations of up to 30 degrees to improve generalization and reduce overfitting. All images were resized to 224×224 pixels to match the input requirements of ResNet-50. The images were then converted into PyTorch tensors and normalized using ImageNet mean and standard deviation values to ensure stable training and consistent input scaling.

Model Description:

Our model is built on ResNet 50. It has 50 layers and 25.6 million parameters (*ResNet50* | *Keras-Team/Keras-Applications* | *DeepWiki*, 2025). We chose this model because the pretrained model already can recognize features like edges, textures, colors, shapes, and patterns. By fine tuning the model we were able to get it to recognize visual patterns with different classes of acne. We replaced the original 1000 class output layer with a new fully connected layer outputting 6 classes for our acne types. All input images were resized to 224×224 pixels and normalized using ImageNet mean and standard deviation values. Training images were augmented with random horizontal/vertical flips and rotations up to 30 degrees to improve generalization. We used the Adam optimizer and Cross-Entropy loss throughout both phases with a batch size of 32. The training for the model was split into two phases.

For the hyperparameters we choose them through research. With research within pytorch, models typically use an image size 224. Num epochs we tested increments of 5 where 30 was big enough to get good accuracy, but it didn't take a very long time to run. The learning rate is .004 because we wanted to have a smaller step size than the default .01 to increase the accuracy, but .001 was too small, so we upped the step size a small bit.

The first phase was freezing the backbone and training over 30 epochs. The goal was to stabilize the head to prevent large gradients from erratically updating the weights. We used a learning rate of 0.004 to ensure stability. After the 30 epochs, we ended up with ~50% validation accuracy.

The second phase was unfreezing the backbone to train the model for another 30 epochs. Thanks to the work from the first phase, it allows the model to fine-tune with stable weights. The second phase is trained with a learning rate of 0.004, which is 10x smaller in the second phase compared to the first phase. This was used to make small updates to adapt the model to subtle changes in color/texture across classes. We reached a final test accuracy of 90.94%.

As a baseline comparison, Phase 1 alone achieved ~47% validation accuracy. We also considered simpler architectures such as a basic CNN trained from scratch, but given our small dataset of 1757 images, transfer learning with ResNet-50 was the more appropriate choice. As another baseline we used a different Pytorch model, ResNet-18. This model was not super trained, meaning we only ran 30 epochs with it and it has less than half the layers than ResNet-50. This baseline model had a final training loss and accuracy, 1.4864 and .4220. For

validation loss and accuracy its 1.4923 and .4269. The full two-phase approach nearly doubled the baseline performance, reaching 90.94% test accuracy.

Results and Discussion:

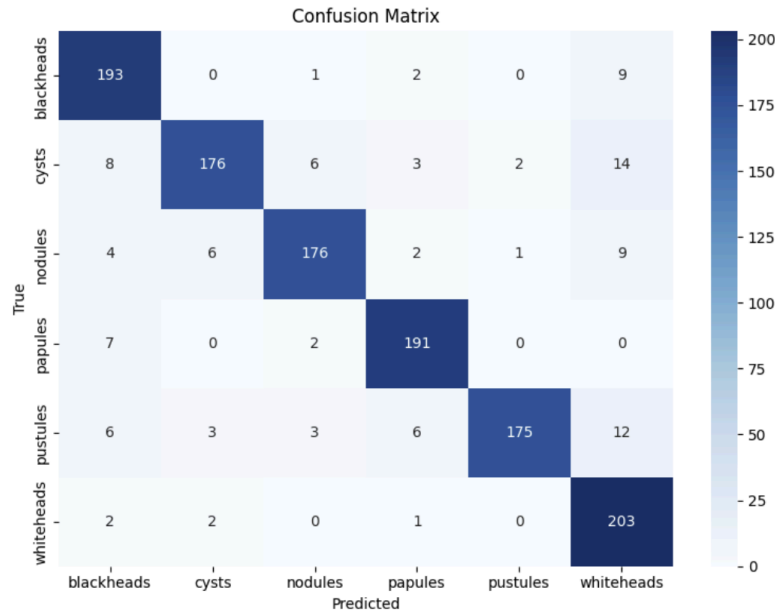


As shown in the first graph, our test accuracy is 0.9094, indicating a fairly accurate model. The orange line is our validation loss. The validation loss measures a trained model's error rate on unseen validation data. This indicates how well it generalizes beyond its training

set. As you can see in the graph, the validation loss is decreasing. This means that our machine learning model is improving its ability to generalize to new data. This also means it will be able to recognize new acne images and generalize them. However, the fluctuations in validation loss indicate some instability in generalization performance. We believe this is caused by our limited dataset size and the noise in validation samples. Despite this, the graph shows the model is learning effectively and maintaining reasonable generalization performance.

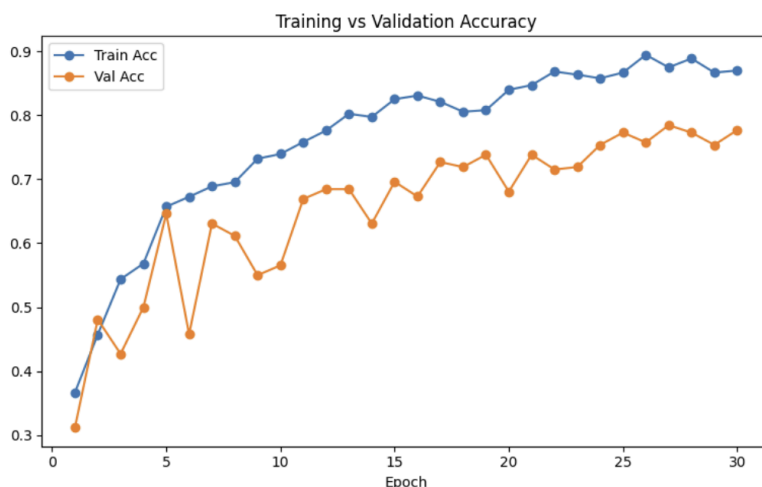
This graph also includes a blue line representing our Train Loss, the metric that measures how poorly the model performed by comparing its predicted outputs to our actual outputs. As you can see, our training loss is decreasing. This means the model is learning acne recognition

patterns and can more accurately determine the different types of acne a person may have.



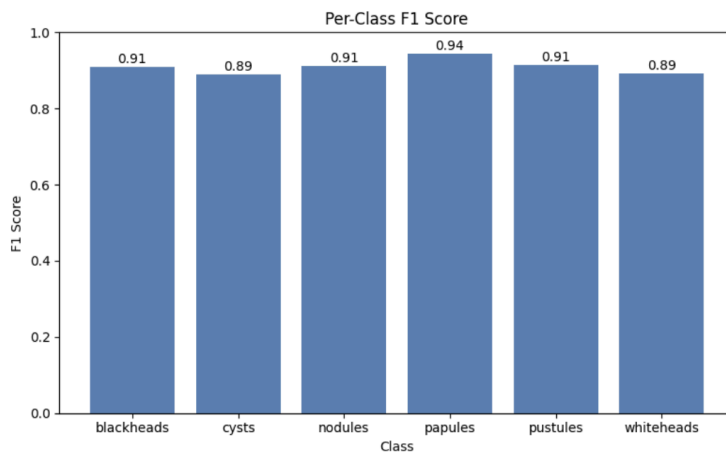
This image represents a confusion matrix. The bluer the square is, the more instances of this there are. The Y-axis of this image refers to the true or “real” cases. The X-axis represents what the system predicted the image was. As we can see, for the majority of the cases, the predicted output matches the true output. When this happens, it indicates that the model is becoming more accurate. We can see that the system correctly identified 193 instances of blackheads with a few misclassifications into other categories.

However, the model exhibits systematic errors among visually similar categories. We can see this occur in cysts and nodules, which are frequently confused with one another. The model might struggle to distinguish between inflamed acne types. This could also be due to a small dataset. But, overall, this model still achieves high accuracy. In the future, this model could be improved by enhancing feature discrimination between similar acne types, potentially through additional training data.



This graph shows our model’s training accuracy and validation accuracy across 30 epochs. The blue line represents training accuracy, and the orange line represents validation accuracy. The x-axis shows the number of epochs,

while the y-axis shows the accuracy. As shown in this graph, both lines generally trend upward as training continues, indicating the model is improving over time. Training accuracy exceeds validation accuracy because the model is optimized directly on the training data. Although the validation accuracy fluctuates across epochs, its overall upward trend suggests that the model is improving its performance on new data. From epoch 0 to around epoch 7 training and validation accuracy grow steadily and grow very closely to another. After epoch 7 training accuracy continues to fluctuate a lot. While training accuracy continues to grow very steadily and then eventually plateaus around 90% accuracy.



These blue bars represent the per-class F1 score of our model for the six acne categories: blackheads, cysts, nodules, papules, pustules, and whiteheads. The x-axis represents the acne classes and the y-axis represents the F1 score. As shown in the graph, the model performs well across

all classes with F1 score ranging from 0.89 to 0.91.

Papules achieved the highest F1 score at 0.94, while cysts and whiteheads had the lowest at 0.89. Blackheads, and nodules, and pustules each achieved 0.91. This indicates that the model performs consistently well for most acne types but cysts may be more difficult to classify because they share visual similarities with other severe acne categories. This result highlights the importance of class-wise evaluation since overall accuracy alone would not show that some classes are more challenging than others.

Conclusion:

Our goal as a group was to build a system that accurately classifies the type of acne the user may have. We built an acne classification system using ResNet-50 that can identify six types of acne from an image, achieving a test accuracy of 90.94%. For this model, we used a two-phase training approach. The first training used only the classification head while keeping the rest of the model frozen. Then we unfroze everything for a second round of fine-tuning. Our baseline ResNet-18 model only hit around 42% accuracy.

Looking at the results by class, the model handles papules best, with an F1 score of 0.94, while cysts and whiteheads are the hardest to get right, at 0.89. This was because cysts and

nodules look very similar, so it makes sense that the model would mix them up. This is backed up by the confusion matrix. One improvement we believe can make this more accurate in the future is training the model on more instances of whiteheads and cysts.

That said, the project has some limitations we want to be upfront about. The dataset is only 1,757 images. We believe this could be why validation loss fluctuated so much during training. However, if we were to keep working on this, we would want to collect more data, especially for the classes the model confuses most. In the future, we would also like to build a frontend that allows users to upload or take photos directly. As of right now, everything runs in the backend. Also, in the future, we would like to get the model to work with full-face images rather than cropped close-ups. This would really help people use the model better day to day.

Reflection and Contribution:

When doing a classification project it's good to look at PA1 as we made a perceptron. A lot of the code was based off that, but with a few different tweaks. Pytorch does a lot of the heavy lifting for you, the library has its own backward function and built in loss functions. Pytorch does a lot of the math that we did in class and does it all for you. Also using a pretrained model helps considerably and saves a lot of time.

Mason: Assisted with producing the code, did research on Resnet 50 and Pytorch to help develop the project and helped out the report.

Before starting this project I had a base understand of the Pytorch library. After doing this project I gained a lot more experience and knowledge about the library, from it's pretrained models, to more methods for transformations like `.Compose()`. Since we have done a classification model in class before I used that as an example to help make the main loop, but everything else was handled by Pytorch, which I thought was really cool.

Ethan: Assisted with producing code, developing important data visualizations, and debugging issues while working in our Kaggle development environment. During the project, I gained a better understanding of how to properly load and connect image datasets within Kaggle, including handling file paths and ensuring the data was correctly processed by our PyTorch pipeline. PyTorch made deep learning extremely fast, which surprised me due to its use of vectorized operations and optimized linear algebra libraries. This resulted in significantly faster performance compared to standard Python implementations.

Alex: Helped with research in the databases, writing the report, helped with debugging code, and related the project to other similar work. I also helped write a lot of the project report. I wrote the introduction and conclusion, image descriptions, and helped with many other writing requirements for this report. From this project, I was able to apply what I learned in my PAs for

this class. I also learned how to use Kaggle and upload datasets. I was able to learn new things about Pytorch and gain a better understanding of what I learned in class and lecture. For example, when I trained the model, I was able to apply my knowledge of overfitting and overshooting to help fix it. I was also able to apply my knowledge for PA2 to actually train the model and apply what I learned about learning rates and epochs. Doing this project was a lot of fun for me. I was able to understand the concepts we learned in class on a much deeper level, and I also learned how to implement databases in code and get real hands-on experience with a coding project.

Jay: Helped with research for the project and helped to write a report.

A.I:

We used Claude to help us get starter code and debug this project. Anytime we got stuck, it helped us and guided us through the process. We also used it to create examples and based some of our code on that. We also used Claude to help explain the different functions used in the examples it provided, as well as any functions in the PyTorch documentation that were confusing.

References:

Anthropic.(3/10/26). Acne Classification Model with Resnet-50. Claude.

Eczema Dataset. (2025). *Acne Dataset*. Roboflow.

<https://universe.roboflow.com/eczema-dataset/acne-gnrti>

MicroAI Admin. (2021, May 13). Deep Learning Models Used in Medical Imaging Analysis.

<https://www.micro.ai/blog/deep-learning-models-in-medical-imaging-and-healthcare-data>

Models and pre-trained weights — Torchvision main documentation. (2024). Pytorch.org.

<https://docs.pytorch.org/vision/main/models.html>

Optimizing Model Parameters — PyTorch Tutorials 2.7.0+cu126 documentation. (2024).

Pytorch.org. https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

ResNet50 | keras-team/keras-applications | DeepWiki. (2025). DeepWiki.

<https://deepwiki.com/keras-team/keras-applications/3.2.1-resnet50>

resnet50 — Torchvision main documentation. (2024). Pytorch.org.

<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html>

Spotscan+, acne-prone skin analysis powered by A.I. (2018). Laroche-Posay.me.

<https://www.laroche-posay.me/en/event/spotscan-plus>